Name: Chandra Prakash D

Reg no: 192524060

Task: Travelling Salesperson Problem (Tsp) Using Dfs In Prolog

1. Introduction

The Travelling Salesperson Problem (TSP) is a well-known problem in Artificial Intelligence. The objective is to find the shortest possible tour that starts from a city, visits every other city exactly once, and returns to the starting city.

In the given graph, there are five cities: A, B, C, D, and E. The numbers on the edges represent the travel costs between connected cities. The graph is undirected, meaning that travel is possible in both directions along each road.

Depth-First Search (DFS) can be used to explore different possible routes. By calculating and comparing their total costs, we can identify the tour with the minimum cost.

2. Prolog Program Using DFS

The following Prolog program uses DFS to generate valid tours,
calculate their costs, and find the minimum-cost tour.

```
% Roads and their
costs road(a, b, 4).
road(a, c, 1). road(a, e,
3). road(b, d, 5).
```

```
road(b, e, 2).
road(c, d, 3).
road(c, e, 2).
road(d, e, 1).

% Allow travel in both directions
edge(X, Y, C) :- road(X, Y, C).
edge(X, Y, C) :- road(Y, X, C).

% DFS: visit every city exactly once
dfs(Current, Start, Visited, CostSoFar, Tour, TotalCost)
:-    length(Visited, 5),    edge(Current, Start,
ReturnCost),    append(Visited, [Start], Tour),
   TotalCost is CostSoFar + ReturnCost.

dfs(Current, Start, Visited, CostSoFar, Tour, TotalCost) :-
edge(Current, Next, C),
   \+ member(Next, Visited),
NewCost is CostSoFar + C,
append(Visited, [Next], NewVisited),
dfs(Next, Start, NewVisited, NewCost,
Tour, TotalCost).

% Generate tours starting from city A
tour(Tour, Cost) :-
```

```
    dfs(a, a, [a], 0, Tour, Cost).

% Find the minimum-cost tour
best_tour(Tour, Cost) :-
findall(C-T, tour(T, C), Tours),
sort(Tours, [Cost-Tour | _]).
```

3. Explanation of the Program

1. **Road facts:** The road/3 predicates define the connections between cities and their costs.
2. **Edge predicate:** The edge/3 rules allow travel in both directions.
3. **DFS traversal:** The dfs/6 predicate explores one route at a time and moves to an unvisited city.
4. **Avoiding repetition:** The member/2 check prevents a city from being visited more than once before the tour is complete.
5. **Returning to the start:** After visiting all five cities, the algorithm checks whether the current city connects back to A.
6. **Cost calculation:** The program adds the cost of every road, including the final return to A.
7. **Finding the best tour:** The best_tour/2 predicate collects all valid tours, sorts them by cost, and returns the minimum-cost tour.

4. Execution and Output

After entering the program into a Prolog interpreter, execute the following query:

```
?- best_tour(Tour, Cost).
```

The minimum-cost result is:

Tour = [a, c, d, e, b, a]

Cost = 11

The corresponding route is:

A → C → D → E → B → A

5. Calculation of the Optimal Tour Cost

The edge costs for the selected tour are:

- A to C = 1
- C to D = 3
- D to E = 1
- E to B = 2
- B to A = 4

The total cost is calculated as follows:

[

{Total Cost}=1+3+1+2+4

]

[

{Total Cost}=11}

]

Thus, the minimum-cost tour is 11 cost units.

6. Advantages of DFS for TSP

- **Systematic exploration:** DFS explores possible routes one at a time.

- **Avoids repeated cities:** The visited list ensures that each city appears only once before returning to the starting city.
- **Calculates route costs:** It adds the costs of all roads to evaluate each valid tour.
- **Finds the optimal tour:** By comparing all valid tours, the program identifies the minimum-cost solution.

However, DFS may require considerable time for large problems because the number of possible tours increases rapidly as the number of cities grows.

7. Conclusion

The Travelling Salesperson Problem can be solved using Depth-First Search in Prolog by exploring valid routes, ensuring that every city is visited exactly once, and returning to the starting city.

For the given graph, the optimal tour is:

A → C → D → E → B → A

The total path cost is:

[

{1+3+1+2+4=11}

]

Therefore, the minimum travel cost is **11 cost units**. This example demonstrates how DFS and Prolog can be used to solve a routeoptimization problem by systematically generating and comparing possible tours.